

TECHNICAL DEEP DIVE

How blindnavandroid Achieves Sub-Meter Walking Navigation

A line-by-line explanation of the ARCore Geospatial (VPS) pipeline, heading fusion, route geometry, and spoken guidance that keep a walker on the sidewalk where Google Maps drifts into the street.

Codebase `com.google.ar.core.codelabs.hellogeospatial`

Platform Android · ARCore Geospatial API · Kotlin

Audience Engineers extending or reviewing the navigation core

Contents

1. Executive summary — why the dot doesn't drift
2. GPS vs. VPS: the root cause of the accuracy gap
3. System architecture
4. Enabling Geospatial mode & the session lifecycle
5. The per-frame render loop (`onDrawFrame`)
6. Heading fusion: the circular moving average
7. Drawing the position on the 2D map
8. The routing pipeline: destination → polyline
9. Polyline decoding
10. Route geometry & on-route detection (the distance algorithm)
11. Turn-by-turn guidance geometry (`ArrowAlignedGuide`)
12. Spoken guidance (`SpeechGuide` / TTS)
13. The OCR side-channel
14. Accuracy budget, failure modes & how to tighten it
15. Appendix: end-to-end data-flow trace

1 Executive summary — why the dot doesn't drift

The blue dot in Google Maps is positioned from **GNSS/GPS radio signals** fused with Wi-Fi and cell information. In a city those radio signals bounce off building faces ("urban-canyon multipath"), so the raw fix is routinely **5–20 m off**. Maps hides some of that by *snapping* the dot to the nearest road centerline — which is precisely why it appears to jump from the sidewalk into the middle of the street.

This app never asks the operating system where you are. Instead it turns on one capability — ARCore **Geospatial mode** — and reads position from a completely different source: **VPS, the Visual Positioning Service**. VPS looks through the phone's camera, matches what it sees against Google's global 3D model of the world (built from years of Street View imagery), and solves for where the camera physically is relative to that known geometry. That is a computer-vision pose estimate, not a radio triangulation, and in VPS-covered areas it delivers roughly **sub-meter horizontal accuracy and a few degrees of heading accuracy**.

Everything else in the app — the moving map marker, the walking route, the "turn left in 5 meters", the "you are drifting to the right" warnings — is built on top of that one high-accuracy position/heading stream. The remainder of this document walks every stage of that pipeline, in source order, and explains the math.

The one-line answer. It is not a better GPS algorithm. It is *not GPS at all*: camera-based visual localization against Google's Street-View-derived 3D map (ARCore Geospatial / VPS), fused with the phone's inertial sensors, which is inherently sub-meter where GPS is inherently several meters.

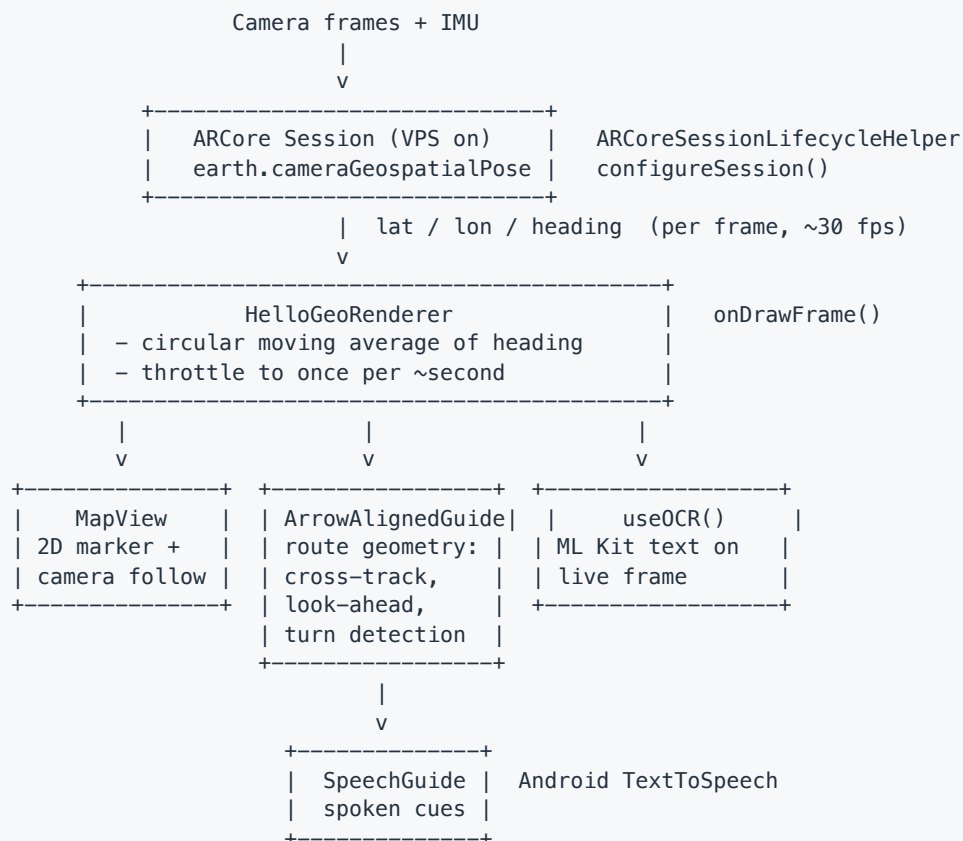
2 GPS vs. VPS: the root cause of the accuracy gap

	Google Maps blue dot (GPS)	This app (ARCore VPS)
Primary signal	GNSS radio + Wi-Fi + cell	Camera imagery matched to a 3D world model
Typical urban error	5–20 m horizontal	~0.5–1 m horizontal
Heading source	Magnetometer / inferred from motion	Visual pose + IMU, ~1–5°
Main failure mode	Multipath in "urban canyons"	No VPS coverage / featureless / dark scenes
Cosmetic fix used	Snap-to-road (causes the street drift)	None needed — reports true position

Because VPS derives position from *what the camera literally sees*, the reported latitude/longitude stays glued to the true physical location of the phone — on the sidewalk — and does not need road-snapping. That single difference is the entire "how is it doing that" the user asked about. The rest is engineering to make that stream usable for a walker (and specifically a blind walker).

3 System architecture

The app is a single `Activity` wiring together a handful of lifecycle-aware components. The diagram shows the flow of a position sample from the camera all the way to the user's ear.



Routing (one-shot, on destination set):
 destination text/voice --> Geopify geocode --> Google Directions API
 --> encoded polyline --> decodePoly() --> viewPointList (List<LatLng>)

Roles at a glance:

- `HelloGeoActivity` — entry point; builds the components, enables Geospatial mode, wires speech-to-text for destination entry.
- `ARCoreSessionLifecycleHelper` — creates/resumes/pauses/closes the ARCore session safely across the Android lifecycle.
- `HelloGeoRenderer` — the heartbeat; runs every frame, reads the geospatial pose, smooths heading, drives the map and the guide.
- `MapView` — the 2D Google Map: user marker, route polyline, camera that follows heading.
- `ArrowAlignedGuide` — pure route geometry & the spoken-turn state machine (no reliance on Google's turn text).
- `SpeechGuide` — a thin, thread-safe wrapper over Android TextToSpeech tuned for navigation audio.

4 Enabling Geospatial mode & the session lifecycle

Everything hinges on one configuration flag. Without it, `session.earth` is unavailable and there is no VPS position at all.

`HelloGeoActivity.kt` — the switch that turns on VPS

```

172 fun configureSession(session: Session) {
173     // TODO: Configure ARCore to use GeospatialMode.ENABLED.
174     session.configure(
175         session.config.apply {
176             // Enable Geospatial Mode.
177             geospatialMode = Config.GeospatialMode.ENABLED
178         }
179     )
180 }

```

`GeospatialMode.ENABLED` tells ARCore to start the VPS localization pipeline: it begins streaming visual features to Google's localization service and fusing the returned world-anchored pose with on-device visual-inertial odometry (SLAM). From that point on, `session.earth.cameraGeospatialPose` yields Earth-referenced latitude, longitude, altitude, and heading.

This `configureSession` is registered as `beforeSessionResume` so it runs every time the session is about to resume — the lifecycle helper below guarantees a session exists and is configured before `resume()`.

ARCoreSessionLifecycleHelper.kt – safe creation & resume

```

65 private fun tryCreateSession(): Session? {
66     // The app must have been given the CAMERA permission. If we don't have it yet, request it.
67     if (!GeoPermissionsHelper.hasGeoPermissions(activity)) {
68         GeoPermissionsHelper.requestPermissions(activity)
69         return null
70     }
71
72     return try {
73         // Request installation if necessary.
74         when (ArCoreApk.getInstance().requestInstall(activity, !installRequested!!)) {
75             ArCoreApk.InstallStatus.INSTALL_REQUESTED -> {
76                 installRequested = true
77                 // tryCreateSession will be called again, so we return null for now.
78                 return null
79             }
80             ArCoreApk.InstallStatus.INSTALLED -> {
81                 // Left empty; nothing needs to be done.
82             }
83         }
84
85         // Create a session if Google Play Services for AR is installed and up to date.
86         Session(activity, features)
87     } catch (e: Exception) {
88         exceptionCallback?.invoke(e)
89         null
90     }
91 }
92
93 override fun onResume(owner: LifecycleOwner) {
94     val session = this.session ?: tryCreateSession() ?: return
95     try {
96         beforeSessionResume?.invoke(session)
97         session.resume()
98         this.session = session
99     } catch (e: CameraNotAvailableException) {
100         exceptionCallback?.invoke(e)
101     }
102 }

```

Key correctness points here:

- **Permission & install gating.** `tryCreateSession()` returns `null` (no crash) if the CAMERA permission is missing or if Google Play Services for AR needs installing/updating; it is retried on the next resume once the user complies.
- **Configure-then-resume ordering.** `beforeSessionResume?.invoke(session)` (which is `configureSession`) runs *before* `session.resume()`, so VPS is enabled the moment the camera starts.
- **Deterministic teardown.** `onPause` pauses and `onDestroy` closes the session, releasing the camera and native VPS resources — important because the camera is a shared, exclusive device.

5 The per-frame render loop

`onDrawFrame` is the app's heartbeat — ARCore calls it once per camera frame (~30 fps, throttled to the camera rate because the session runs in blocking update mode). Each call advances the AR frame, then — only when the Earth subsystem is confidently `TRACKING` — reads a fresh geospatial pose and fans it out.

HelloGeoRenderer.kt — the geospatial branch of `onDrawFrame`

```

224     val earth = session.earth
225     if (earth?.trackingState == TrackingState.TRACKING) {
226         val cameraGeospatialPose = earth.cameraGeospatialPose
227
228         headingValues.addLast(cameraGeospatialPose.heading)
229         avgHeading += cameraGeospatialPose.heading
230
231         if(headingValues.size > oneSecond){
232             avgHeading -= headingValues.removeFirst()
233         }
234
235         val cosSum = headingValues.sumOf { cos(Math.toRadians(it)) }
236         val sinSum = headingValues.sumOf { sin(Math.toRadians(it)) }
237
238         var avgHeading = Math.toDegrees(atan2(sinSum / headingValues.size, cosSum /
239 headingValues.size))
240
241         if (avgHeading > 180) {
242             avgHeading -= 360.0
243         }
244
245         activity.view.mapView?.updateMapPosition(
246             latitude = cameraGeospatialPose.latitude,
247             longitude = cameraGeospatialPose.longitude,
248             heading = avgHeading,
249             isClick
250         )
251
252         if(activity.view.ocrIsOn){
253             useOCR(frame)
254         }
255
256         if(isClick && curFrame++ == oneSecond){
257             val curLatLng = LatLng(cameraGeospatialPose.latitude, cameraGeospatialPose.longitude)
258             val path = activity.view.mapView?.viewPointList.orEmpty()
259
260             if (path.size >= 2) {
261                 val status = activity.navGuide.update(
262                     user = curLatLng,
263                     headingDeg = avgHeading,
264                     path = path,
265                 )
266
267                 activity.runOnUiThread {
268                     if (status != null) {
269                         activity.view.distance_text.text = status.statusText
270                         if (status.statusText == "Arrived" || status.distToDestMeters < 1.5) {
271                             Toast.makeText(activity, "Arrived", Toast.LENGTH_LONG).show()
272                             activity.reset()
273                         } else if (!status.onRoute) {
274                             showSnackbar(activity.findViewById(android.R.id.content), "Veering off")
275                         }
276                     }
277                 }
278                 curFrame = 0
279             }
280         }

```

Walking through it:

- **Gate on tracking (line 225).** Nothing downstream runs unless `earth.trackingState == TRACKING`. This is what prevents the app from ever acting on a low-confidence or not-yet-localized pose — the equivalent of Maps showing a "searching..." grey dot, but enforced in code.
- **Read the pose (line 226).** `cameraGeospatialPose` is the VPS output: `.latitude`, `.longitude`, `.heading`, `.altitude`.
- **Smooth the heading (lines 228–242).** Raw per-frame heading is jittery, so it is run through a circular moving average — detailed in the next section.
- **Push to the map (line 244).** `updateMapPosition` moves the 2D marker and, when navigating, rotates the map to the walker's facing.
- **Throttle the guidance (line 255).** `curFrame++ == oneSecond` (`oneSecond = 29`) fires the navigation update roughly once per second rather than every frame — guidance geometry and speech don't need 30 Hz, and this keeps the spoken cues calm.
- **Arrival & off-route UX (lines 266–276).** On the UI thread it updates the status text, declares arrival under 1.5 m, and surfaces a "Veering off" snackbar when the guide reports the walker has left the route corridor.

Why gate everything on `TRACKING`? A VPS pose that hasn't converged can be tens of meters wrong for a moment. By refusing to move the marker, speak, or evaluate the route until the state is `TRACKING`, the app trades a short startup delay for never emitting a confidently-wrong instruction to a blind user.

6 Heading fusion: the circular moving average

Heading is an **angle**, so you cannot average it arithmetically: the mean of 359° and 1° is 180° (pointing backwards) when the true answer is 0°. The renderer solves this correctly by averaging the *unit vectors* of the recent headings and taking the resulting vector's angle — the standard "mean of circular quantities".

HelloGeoRenderer.kt – windowed circular mean of heading

```

228     headingValues.addLast(cameraGeospatialPose.heading)
229     avgHeading += cameraGeospatialPose.heading
230
231     if(headingValues.size > oneSecond){
232         avgHeading -= headingValues.removeFirst()
233     }
234
235     val cosSum = headingValues.sumOf { cos(Math.toRadians(it)) }
236     val sinSum = headingValues.sumOf { sin(Math.toRadians(it)) }
237
238     var avgHeading = Math.toDegrees(atan2(sinSum / headingValues.size, cosSum /
    headingValues.size))
239
240     if (avgHeading > 180) {
241         avgHeading -= 360.0
242     }
243
244     activity.view.mapView?.updateMapPosition(
245         latitude = cameraGeospatialPose.latitude,
246         longitude = cameraGeospatialPose.longitude,
247         heading = avgHeading,
248         isClick
249     )

```

Mechanics:

- **Sliding window.** `headingValues` is an `ArrayDeque` holding up to `oneSecond = 29` recent samples (~1 s at 30 fps). Older samples are dropped from the front.
- **Vector sum.** Each heading θ contributes $(\cos \theta, \sin \theta)$; summing these and taking `atan2( $\Sigma \sin$ ,  $\Sigma \cos$ )` gives the average direction with no wraparound error.
- **Normalize to $\pm 180^\circ$.** The result is folded into the range the map and guide expect (`if (avgHeading > 180) avgHeading -= 360`).

The net effect: a stable, low-lag facing direction. This matters more here than in a typical map app, because the **heading** — not just position — drives the spoken "turn left / turn right" instructions. A jittery heading would make the guide stutter contradictory turn cues; the smoothing is what makes the voice guidance trustworthy.

Note. There is a deliberate shadowing here: the running scalar `avgHeading` field is maintained but then a local `avgHeading` (the circular mean) is what's actually used and passed downstream. The circular-mean local is the correct value; the scalar accumulator is vestigial.

7 Drawing the position on the 2D map

`updateMapPosition` receives the smoothed pose and reconciles it with the Google Map. It is careful not to fight the user's own map gestures.

```
MapView.kt — marker + heading-locked camera follow
```

```

141 fun updateMapPosition(latitude: Double, longitude: Double, heading: Double, isClick: Boolean)
142 {
143     val position = LatLng(latitude, longitude)
144     activity.runOnUiThread {
145         // If the map is already in the process of a camera update, then don't move it.
146         if (!cameraIdle) {
147             return@runOnUiThread
148         }
149         cameraMarker.isVisible = true
150         cameraMarker.position = position
151         cameraMarker.rotation = heading.toFloat()
152
153         if(setFirst){
154             val cameraPositionBuilder: CameraPosition.Builder = if (!setInitialCameraPosition) {
155                 // Set the camera position with an initial default zoom level.
156                 setInitialCameraPosition = true
157                 CameraPosition.Builder().zoom(21f).target(position)
158             } else {
159                 // Set the camera position and keep the same zoom level.
160                 CameraPosition.Builder()
161                     .zoom(googleMap.cameraPosition.zoom)
162                     .target(position)
163             }
164             googleMap.moveCamera(
165                 CameraUpdateFactory.newCameraPosition(cameraPositionBuilder.build()))
166             setFirst = false
167         }
168         if(isClick){
169             val adjustedHeading = if (heading < 0) heading + 360 else heading
170             googleMap.moveCamera(
171                 CameraUpdateFactory.newCameraPosition(
172                     CameraPosition.Builder()
173                         .target(position)
174                         .zoom(googleMap.cameraPosition.zoom) // Set your preferred zoom level
175                         .bearing(adjustedHeading.toFloat()) // Set the orientation of the camera to
176                         follow the user's heading
177                     .build()
178                 )
179             )
180             googleMap.uiSettings.isCompassEnabled = true
181         }else{
182             googleMap.uiSettings.isCompassEnabled = false
183         }
184     }
185 }

```

- **Don't fight the user (line 145).** If the map camera is mid-move (`!cameraIdle`), the update returns early so a programmatic camera move doesn't yank the view while the user is panning.
- **Marker is the true VPS position (lines 148–150).** The green navigation marker is placed at the VPS lat/lng and *rotated* to the smoothed heading — so the arrow points where the walker actually faces.
- **First-fix framing (lines 152–166).** On the first sample it zooms in tight (`zoom(21f)` — building-level) and centers; afterward it preserves the user's chosen zoom.
- **Heading-up navigation (lines 167–178).** Once a destination is set (`isClick`), the camera `bearing` is locked to the walker's heading, giving an egocentric "the world rotates around me" view and enabling the compass. The heading is wrapped to [0,360) for the Maps API.

8 The routing pipeline: destination → polyline

Routing is a one-shot operation triggered when the user sets a destination — either by tapping the map, or by typing/speaking a place name. The accuracy story is entirely on the *positioning* side; the route itself is a standard Google Directions walking route. Here is the tap path.

```
HelloGeoRenderer.kt — onMapClick: build the route & anchor
```

```

411 fun onMapClick(latLng: LatLng) {
412     viewPoints.clear()
413     activity.view.mapView?.eraseLines()
414     activity.navGuide.reset()
415     isClick = false
416
417     // To check the earth is available or not
418     val earth = session?.earth ?: return
419     if (earth.trackingState != TrackingState.TRACKING) {
420         return
421     }
422
423     //if the tracking state is tracking then...
424     if (earth.trackingState == TrackingState.TRACKING) {
425         val cameraGeospatialPose = earth.cameraGeospatialPose
426         val curLatitude = cameraGeospatialPose.latitude
427         val curLongitude = cameraGeospatialPose.longitude
428
429         val url:String = getDirectionsUrl(LatLng(curLatitude,curLongitude),latLng)!!
430         fetchJson(url) { stepList ->
431
432             viewPoints = stepList.toMutableList()
433             activity.view.mapView?.erasePoints()
434
435             for(item in viewPoints){
436                 val poly = decodePoly(item.polyline.points)
437                 activity.view.mapView?.addViewPoints(poly)
438
439                 activity.view.mapView?.addViewPointMarker(LatLng(item.end_location.lat,item.end_location.lng))
440                 // blueWaypoint.add(LatLng(item.end_location.lat,item.end_location.lng))
441             }
442             activity.view.mapView?.drawPolyline()
443             isClick = true
444
445             val path = activity.view.mapView?.viewPointList.orEmpty()
446             if (path.size >= 2) {
447                 activity.navGuide.onRouteReady(
448                     path = path,
449                     user = LatLng(curLatitude, curLongitude),
450                     headingDeg = cameraGeospatialPose.heading,
451                 )
452             }
453         }
454
455         earthAnchor?.detach()
456         // Place the earth anchor at the same altitude as that of the camera to make it easier to
457         view.
458         val altitude = earth.cameraGeospatialPose.altitude - 1
459         // The rotation quaternion of the anchor in the East-Up-South (EUS) coordinate system.
460         val qx = 0f
461         val qy = 0f
462         val qz = 0f
463         val qw = 1f
464         earthAnchor =
465             earth.createAnchor(latLng.latitude, latLng.longitude, altitude, qx, qy, qz, qw)
466         activity.view.mapView?.earthMarker?.apply {
467             position = latLng
468             isVisible = true
469         }
470     }
471 }

```

Step by step:

- **Origin = VPS position (lines 425–427).** The route's origin is the current `cameraGeospatialPose`, i.e. the sub-meter VPS fix — not a GPS fix. A precise origin is what lets the corridor math later be meaningful at a 3–5 m scale.
- **Ask Google Directions (lines 429–430).** `getDirectionsUrl` builds a walking-mode request; `fetchJson` executes it off the main thread.
- **Decode & store the path (lines 435–441).** Each step's encoded `polyline` is expanded to `LatLng` points and appended to `viewPointList`, then drawn as a red polyline.
- **Hand the geometry to the guide (lines 444–451).** Once there are ≥ 2 points, `navGuide.onRouteReady` primes the turn-by-turn state machine with the path and the current pose.
- **Drop a world anchor (lines 455–464).** An ARCore `Earth` anchor is created at the destination lat/lng so the 3D marker is pinned to real-world geometry, tracked as ARCore refines its estimate.

The request URL is deliberately simple — walking mode, JSON output:

HelloGeoRenderer.kt – Directions request builder

```
535 private fun getDirectionsUrl(origin: LatLng, dest: LatLng): String? {
536     val stringOrigin = "origin="+origin.latitude+","+origin.longitude
537
538     val stringDest = "destination="+dest.latitude+","+dest.longitude
539
540     val mode = "mode=walking"
541
542     val parameters = "$stringOrigin&$stringDest&$mode"
543
544     val output = "json"
545
546     // API key
547     return "https://maps.googleapis.com/maps/api/directions/$output?
    $parameters&waypoints&key=AIzaSyB7BBKr7-7-s5TuK4pN-jhuJa2JxaExmmo"
548 }
```

The typed/spoken path (in `HelloGeoView`) adds a geocoding hop: it reverse-geocodes the current VPS position to a postcode (Geoapify), then forward-geocodes the spoken place name constrained to a 5 km circle around the user, and finally calls the same `onMapClick` with the resolved coordinate.

HelloGeoView.kt – destination submit (voice/text)

```

79 fun setOnClickListener(){
80     submitButton.setOnClickListener {
81         val placeName = editText.text.toString()
82         val earth = session?.earth ?: return@setOnClickListener
83         if (placeName.isNotEmpty()) {
84             val cameraGeospatialPose = earth.cameraGeospatialPose
85             val curLatitude = cameraGeospatialPose.latitude
86             val curLongitude = cameraGeospatialPose.longitude
87             val client = OkHttpClient().newBuilder()
88                 .build()
89             // API key
90             val postcodeUrl = "https://api.geoapify.com/v1/geocode/reverse?
lat=$curLatitude&lon=$curLongitude&format=json&apiKey=4a322939529a452897cc7d4d1433aa3d"
91             Log.i(TAG, "setOnClickListener: $postcodeUrl")
92             getPostCode(postcodeUrl){ postcode->
93                 getGeocoding(LatLng(curLatitude,curLongitude),placeName,postcode){ latLng->
94                     activity.render.onMapClick(latLng)
95                 }
96             }
97         }
98     }
99 }
100 ocrButton.setOnClickListener{
101     ocrIsOn = !ocrIsOn
102     if(ocrIsOn) {
103         ocrButton.text = "OCR ON"
104         ocrButton.setBackgroundResource(R.color.red)
105     } else {
106         ocrButton.text = "OCR OFF"
107         ocrButton.setBackgroundResource(R.color.blue)
108     }
109 }
110
111 }

```

9 Polyline decoding

Google returns each route step's geometry as an **encoded polyline** — a compact ASCII string.

`decodePoly` is the standard Google algorithm: it reads variable-length, zig-zag-encoded, 5-decimal-place deltas and reconstructs the `LatLng` vertices.

HelloGeoRenderer.kt — encoded-polyline decoder

```

502 private fun decodePoly(encoded: String): List<LatLng> {
503     val poly = ArrayList<LatLng>()
504     var index = 0
505     val len = encoded.length
506     var lat = 0
507     var lng = 0
508     while (index < len) {
509         var b: Int
510         var shift = 0
511         var result = 0
512         do {
513             b = encoded[index++].toInt() - 63
514             result = result or (b and 0x1f shl shift)
515             shift += 5
516         } while (b >= 0x20)
517         val dlat = if (result and 1 != 0) (result shr 1).inv() else result shr 1
518         lat += dlat
519         shift = 0
520         result = 0
521         do {
522             b = encoded[index++].toInt() - 63
523             result = result or (b and 0x1f shl shift)
524             shift += 5
525         } while (b >= 0x20)
526         val dlng = if (result and 1 != 0) (result shr 1).inv() else result shr 1
527         lng += dlng
528         val p = LatLng(lat.toDouble() / 1E5,
529             lng.toDouble() / 1E5)
530         poly.add(p)
531     }
532     return poly
533 }

```

For each coordinate it accumulates 5-bit groups until the continuation bit clears, undoes the zig-zag ((result shr 1).inv() for negatives), scales by 1e-5, and adds the delta to the running lat/lng. These vertices become the corridor centerline the guidance geometry measures against.

10 Route geometry & on-route detection (the distance algorithm)

This is the heart of "am I still on the sidewalk path?" Given the precise VPS position and the decoded route, the app computes the walker's **perpendicular (cross-track) distance** to the nearest route segment and compares it against a tolerance. Because the position is sub-meter, the tolerance can be a tight few meters — impossible with GPS noise.

HelloGeoRenderer.kt — isInRoute: nearest-segment corridor test

```

363 private fun isInRoute(i1: List<LatLng>, curPosition: LatLng): Boolean {
364     // Iterate over each pair of points in the list
365     var minD = 90.00
366     for (i in 0 until i1.size - 1) {
367         val start = i1[i]
368         val end = i1[i + 1]
369         var distance = 100.0
370
371         var startToCur = getDistanceBetween(start, curPosition)
372         var endToCur = getDistanceBetween(end, curPosition)
373         var startToEnd = getDistanceBetween(start, end)
374
375         val angleA = findAngle(startToEnd, endToCur, startToCur) // start 的角度
376         val angleB = findAngle(startToEnd, startToCur, endToCur) // end 的角度
377
378         if (angleA < PI/2 && angleB < PI/2) {
379             distance = distanceFromPointToLine(startToCur, endToCur, startToEnd)
380             minD = min(minD, distance)
381             if (minD <= FIX_DISTANCE && !minD.isNaN()) {
382                 minDis = (minD * 100.0).roundToInt() / 100.0
383                 return true
384             }
385         }
386     }
387     minDis = (minD * 100.0).roundToInt() / 100.0
388     return false
389 }

```

For every consecutive pair of route points it forms a triangle between the segment endpoints and the walker, and uses the triangle's geometry to get the perpendicular distance:

HelloGeoRenderer.kt – distanceFromPointToLine (Heron's formula)

```

346 private fun distanceFromPointToLine(startToCur: Double, endToCur: Double, startToEnd:
Double): Double {
347
348     if (startToCur < FIX_DISTANCE * 1.2 && endToCur < FIX_DISTANCE * 1.2) {
349         return startToCur
350     }
351
352     return if (startToCur + endToCur <= startToEnd) { // 如果起点到当前点的距离加上终点到当前点的距离小于等于
起点到终点返回 0
353         0.0
354     } else {
355         var p = (startToEnd + startToCur + endToCur) / 2
356         var q = max(p * (p - startToEnd) * (p - startToCur) * (p - endToCur), 0.0)
357         var s = sqrt(q)
358         return 2 * s / startToEnd
359     }
360 }

```

The math, decoded:

- **Near-vertex short-circuit (line 348).** If the walker is within `FIX_DISTANCE*1.2` of *both* endpoints, just return the start-to-current distance — near a waypoint the perpendicular is meaningless.
- **Projection falls outside the segment (line 352).** If `startToCur + endToCur ≤ startToEnd` (degenerate/collinear-beyond case) it returns 0.

- **Heron's formula (lines 355–358).** Otherwise it computes the triangle area from the three side lengths via the semi-perimeter `p`, then `perpendicular = 2·area / base`, where the base is the route segment `startToEnd`. That perpendicular is the cross-track distance.

The `isInRoute` wrapper additionally guards with the two base angles (`angleA`, `angleB`): only when both are acute is the foot of the perpendicular actually on the segment, so the perpendicular distance is valid. It keeps the minimum across all segments and declares "on route" when that minimum is within `FIX_DISTANCE (3 m)`. The great-circle distance between any two coordinates uses the spherical law of cosines:

HelloGeoRenderer.kt — great-circle distance (meters)

```
569 private fun getDistanceBetween(curPosition: LatLng, waypoint: LatLng): Double{
570
571     val la1= rad(curPosition.latitude)
572     val lo1= rad(curPosition.longitude)
573     val la2= rad(waypoint.latitude)
574     val lo2= rad(waypoint.longitude)
575     val d = 6367 * acos(cos(la1) * cos(la2) * cos(lo2-lo1) + sin(la1) * sin(la2))
576     return d*1000
577 }
578
579 private fun rad(d:Double): Double{
580     return d*Math.PI/180.0
581 }
```

(6367 is Earth's radius in km; the result is scaled to meters.) The same formula appears, hardened with a `coerceIn(-1.0, 1.0)` domain clamp, inside `ArrowAlignedGuide` — the clamp prevents `acos` from returning `NaN` on tiny floating-point overshoots for near-zero distances.

11 Turn-by-turn guidance geometry

`ArrowAlignedGuide` is a self-contained module that converts (position, heading, path) into human instructions. Notably it **ignores Google's turn text** and derives turns purely from geometry — so guidance stays correct even when the walker is between the coarse waypoints Google provides. Its tuning constants define the whole behavior:

ArrowAlignedGuide.kt — corridor & turn tuning

```
17 class ArrowAlignedGuide(
18     private val speech: SpeechGuide,
19     private val corridorMeters: Double = 5.0,
20     private val lookAheadMeters: Double = 12.0,
21     private val periodicIntervalMs: Long = 10_000L,
22     private val deviationIntervalMs: Long = 5_000L,
23 ) {
24     companion object {
25         private val TURN_THRESHOLDS_M = listOf(15, 10, 5)
26         private const val TURN_NOW_M = 2.5
27         private const val WAYPOINT_ADVANCE_M = 4.0
28         private const val STRAIGHT_DEG = 20.0
29         private const val TURN_HINT_DEG = 35.0
30     }
```

When a route is set, it snaps to the nearest segment and speaks an initial summary:

ArrowAlignedGuide.kt – onRouteReady

```
61 fun onRouteReady(path: List<LatLng>, user: LatLng, headingDeg: Double) {
62     reset()
63     if (path.size < 2) return
64     segmentIndex = nearestSegmentIndex(path, user)
65     routeStarted = true
66     val dest = path.last()
67     val distDest = distanceMeters(user, dest)
68     val bearingDest = absoluteBearingDeg(user, dest)
69     val lookAhead = lookAheadPoint(path, user, segmentIndex, lookAheadMeters)
70     val rel = relativeBearingDeg(headingDeg, absoluteBearingDeg(user, lookAhead))
71     val turnPhrase = turnInstruction(rel, includeDegrees = true)
72     speech.speak(
73         "Route set. Destination ${metersSpoken(distDest)} away, bearing
    ${bearingDest.roundToInt()} degrees. $turnPhrase",
74         flush = true
75     )
76     lastPeriodicSpeakMs = System.currentTimeMillis()
77 }
```

Then, once per second, `update` recomputes the full situational picture:

ArrowAlignedGuide.kt – per-tick update

```

79 fun update(user: LatLng, headingDeg: Double, path: List<LatLng>, nowMs: Long =
System.currentTimeMillis()): Status? {
80     if (!routeStarted || path.size < 2) return null
81
82     advanceSegment(path, user)
83
84     val dest = path.last()
85     val lookAhead = lookAheadPoint(path, user, segmentIndex, lookAheadMeters)
86     val distNext = distanceMeters(user, lookAhead)
87     val distDest = distanceMeters(user, dest)
88     val bearingDest = absoluteBearingDeg(user, dest)
89     val bearingPath = absoluteBearingDeg(user, lookAhead)
90     val relPath = relativeBearingDeg(headingDeg, bearingPath)
91
92     val (segStart, segEnd) = currentSegment(path)
93     val crossTrack = crossTrackDistanceMeters(user, segStart, segEnd)
94     val onRoute = crossTrack <= corridorMeters && isAlongsideSegment(user, segStart, segEnd)
95     val sideLeft = isLeftOfSegment(user, segStart, segEnd)
96
97     if (distDest < 1.5) {
98         speech.speak("Arrived at destination.", flush = true)
99         return Status(
100             statusText = "Arrived",
101             crossTrackMeters = crossTrack,
102             distToNextMeters = distNext,
103             distToDestMeters = distDest,
104             destBearingDeg = bearingDest,
105             relativeToPathDeg = relPath,
106             onRoute = true,
107         )
108     }
109
110     if (!onRoute) {
111         offRouteStreak++
112         if (offRouteStreak >= 2 && nowMs - lastDeviationSpeakMs >= deviationIntervalMs) {
113             val side = if (sideLeft) "left" else "right"
114             speech.speak("You are drifting to the $side of the path.", flush = true)
115             lastDeviationSpeakMs = nowMs
116         }
117     } else {
118         offRouteStreak = 0
119         announceTurnThresholds(relPath, distNext)
120     }
121
122     if (nowMs - lastPeriodicSpeakMs >= periodicIntervalMs) {
123         val face = turnInstruction(relPath, includeDegrees = true)
124         speech.speak(
125             "Next waypoint ${metersSpoken(distNext)}. " +
126             "Destination bearing ${bearingDest.roundToInt()} degrees, ${metersSpoken(distDest)}.
" +
127             face
128         )
129         lastPeriodicSpeakMs = nowMs
130     }

```

The important geometric primitives it composes:

- **Absolute bearing (lines 272–280).** The compass bearing from the walker to a target point — standard `atan2` forward-azimuth formula, normalized to [0,360).

- **Relative bearing (lines 268–270).** Target bearing minus current heading, normalized to [-180,180]: positive means "the path is to your right", negative "to your left". This single signed number becomes "turn left/right".
- **Look-ahead point (lines 198–221).** Rather than aim at the next raw vertex, it walks `LookAheadMeters` (12 m) forward along the polyline from the walker's projected position and aims there — giving smooth "continue straight / gentle right" cues instead of snapping at each vertex.
- **Cross-track & side (lines 323–361).** The same Heron's-formula perpendicular as above, plus a 2D cross-product sign test (`isLeftOfSegment`) to say whether the drift is to the left or right — which is what powers "you are drifting to the right of the path."

ArrowAlignedGuide.kt – bearing primitives

```

268 fun relativeBearingDeg(headingDeg: Double, targetBearingDeg: Double): Double {
269     return normalize180(targetBearingDeg - headingDeg)
270 }
271
272 fun absoluteBearingDeg(from: LatLng, to: LatLng): Double {
273     val lonDiff = Math.toRadians(to.longitude - from.longitude)
274     val lat1 = Math.toRadians(from.latitude)
275     val lat2 = Math.toRadians(to.latitude)
276     val y = sin(lonDiff) * cos(lat2)
277     val x = cos(lat1) * sin(lat2) - sin(lat1) * cos(lat2) * cos(lonDiff)
278     val bearing = Math.toDegrees(atan2(y, x))
279     return (bearing + 360.0) % 360.0
280 }
281
282 fun distanceMeters(a: LatLng, b: LatLng): Double {
283     val la1 = Math.toRadians(a.latitude)
284     val lo1 = Math.toRadians(a.longitude)
285     val la2 = Math.toRadians(b.latitude)
286     val lo2 = Math.toRadians(b.longitude)
287     val d = 6367.0 * acos(
288         (cos(la1) * cos(la2) * cos(lo2 - lo1) + sin(la1) * sin(la2)).coerceIn(-1.0, 1.0)
289     )
290     return d * 1000.0
291 }

```

Finally, the countdown state machine turns geometry into speech without nagging:

ArrowAlignedGuide.kt – announceTurnThresholds

```

150 private fun announceTurnThresholds(relPath: Double, distNext: Double) {
151     // New look-ahead after passing a bend: reset thresholds when distance jumps up.
152     if (distNext > (TURN_THRESHOLDS_M.maxOrNull() ?: 15) + 5) {
153         announcedThresholds.clear()
154         announcedTurnNow = false
155     }
156
157     val absRel = kotlin.math.abs(relPath)
158     // Only countdown turns when the path meaningfully differs from facing.
159     if (absRel < TURN_HINT_DEG) {
160         return
161     }
162
163     for (threshold in TURN_THRESHOLDS_M) {
164         if (distNext <= threshold && threshold !in announcedThresholds) {
165             announcedThresholds.add(threshold)
166             // Skip higher unused thresholds if we entered mid-band.
167             announcedThresholds.addAll(TURN_THRESHOLDS_M.filter { it > threshold })
168             val dir = if (relPath > 0) "right" else "left"
169             speech.speak("Turn $dir in $threshold meters.")
170         }
171     }
172
173     if (distNext <= TURN_NOW_M && !announcedTurnNow) {
174         announcedTurnNow = true
175         val dir = if (relPath > 0) "right" else "left"
176         speech.speak("Turn $dir now.", flush = true)
177     }
178 }

```

- **Only for real turns (line 159).** Countdowns fire only when the upcoming path differs from the current facing by more than `TURN_HINT_DEG` (35°).
- **Debounced thresholds (lines 163–171).** It announces "turn in 15/10/5 meters" at most once each, and marks higher thresholds consumed if the walker enters mid-band — so it never repeats or counts up.
- **Final commit (lines 173–177).** Within `TURN_NOW_M` (2.5 m) it flushes a "turn left/right now."
- **Deviation & periodic cues** (in `update`) are separately rate-limited (`deviationIntervalMs = 5 s`, `periodicIntervalMs = 10 s`) and only trigger deviation warnings after a 2-tick off-route streak to avoid false alarms from a single noisy sample.

12 Spoken guidance (SpeechGuide / TTS)

For a blind-navigation app the audio channel is the product. `SpeechGuide` is a small, thread-safe wrapper over Android `TextToSpeech` tuned for the job.

SpeechGuide.kt — navigation-tuned TTS

```

14 class SpeechGuide(context: Context) : TextToSpeech.OnInitListener {
15     companion object {
16         private const val TAG = "SpeechGuide"
17     }
18
19     private val tts = TextToSpeech(context.applicationContext, this)
20     @Volatile private var ready = false
21     @Volatile private var speaking = false
22
23     override fun onInit(status: Int) {
24         if (status != TextToSpeech.SUCCESS) {
25             Log.e(TAG, "TTS init failed: $status")
26             ready = false
27             return
28         }
29         val result = tts.setLanguage(Locale.US)
30         if (result == TextToSpeech.LANG_MISSING_DATA || result == TextToSpeech.LANG_NOT_SUPPORTED) {
31             tts.language = Locale.getDefault()
32         }
33         tts.setSpeechRate(0.92f)
34         tts.setAudioAttributes(
35             AudioAttributes.Builder()
36                 .setUsage(AudioAttributes.USAGE_ASSISTANCE_NAVIGATION_GUIDANCE)
37                 .setContentType(AudioAttributes.CONTENT_TYPE_SPEECH)
38                 .build()
39         )
40         tts.setOnUtteranceProgressListener(object : UtteranceProgressListener() {
41             override fun onStart(utteranceId: String?) {
42                 speaking = true
43             }
44
45             override fun onDone(utteranceId: String?) {
46                 speaking = false
47             }
48
49             @Deprecated("Deprecated in Java")
50             override fun onError(utteranceId: String?) {
51                 speaking = false
52             }
53         })
54         ready = true
55     }
56
57     fun isSpeaking(): Boolean = speaking
58
59     fun speak(text: String, flush: Boolean = false) {
60         if (!ready || text.isBlank()) return
61         val mode = if (flush) TextToSpeech.QUEUE_FLUSH else TextToSpeech.QUEUE_ADD
62         val params = Bundle()
63         tts.speak(text, mode, params, "nav-#{System.nanoTime()}")
64     }
65
66     fun stop() {
67         if (ready) tts.stop()
68         speaking = false
69     }
70
71     fun shutdown() {
72         ready = false
73         speaking = false
74         tts.stop()
75         tts.shutdown()

```

```
76 }  
77 }
```

- **Navigation audio routing (lines 34–39).** It sets `USAGE_ASSISTANCE_NAVIGATION_GUIDANCE` so the OS ducks other audio (music, podcasts) for cues instead of talking over them — and routes through the navigation audio focus.
- **Slightly slowed rate (line 33).** `0.92x` speech rate for clarity while walking.
- **Flush vs. queue (lines 59–64).** Urgent cues ("turn now", "drifting") pass `flush = true` → `QUEUE_FLUSH`, interrupting stale speech; routine cues append with `QUEUE_ADD`.
- **Readiness & speaking flags (@Volatile).** Guards prevent speaking before TTS init completes and track whether audio is currently playing, across threads.

13 The OCR side-channel

Independent of navigation, an optional OCR mode runs Google ML Kit text recognition on the live camera frame — e.g. to confirm a door number or sign matches the destination. It is a toggle so it never competes with the navigation compute path unless the user asks for it.

HelloGeoRenderer.kt – useOCR: ML Kit on the AR frame

```
312 private fun useOCR(frame: Frame){  
313     try {  
314         val curImage = frame.acquireCameraImage()  
315         val recognizer = TextRecognition.getClient(TextRecognizerOptions.DEFAULT_OPTIONS)  
316         val result = recognizer.process(curImage,0)  
317         .addOnSuccessListener { visionText ->  
318             var resultText = visionText.text  
319             // Log.i(TAG, "useOCR: resultText $resultText")  
320             resultText = resultText.lowercase()  
321             val addString = activity.view.editText.text.toString()  
322             if(addString != null && addString != ""){  
323                 val charArray= addString.split(" ")  
324                 // Log.i(TAG, "useOCR: resultText ${charArray.toString()}")  
325                 for(i in charArray){  
326                     if(resultText.contains(i.lowercase())){  
327                         // Log.i(TAG, "useOCR:++++++ $i Find")  
328                         showSnackBar(activity.findViewById(android.R.id.content), "Find!!")  
329                         // activity.view.find_text.text = String.format("find")  
330                     }  
331                 }  
332             }  
333             curImage.close()  
334         }  
335         .addOnFailureListener { e ->  
336             Log.e(TAG, "onDrawFrame: $e", )  
337         }  
338     }catch (e: NotYetAvailableException){  
339         Log.e(TAG, "onDrawFrame: $e", )  
340     }catch (e:ResourceExhaustedException){  
341         Log.e(TAG, "onDrawFrame: $e", )  
342     }  
343 }
```

It acquires the current camera image, runs the on-device Latin text recognizer, lowercases the result, and checks whether any word of the target string appears — announcing "Find!!" on a match. Exceptions for

frames that aren't ready yet (`NotYetAvailableException`) or resource pressure are swallowed so OCR never destabilizes the render loop.

14 Accuracy budget, failure modes & how to tighten it

Why the end-to-end result is sub-meter:

- VPS gives a ~0.5–1 m position and few-degree heading in covered areas.
- The `TRACKING` gate discards low-confidence poses entirely.
- Circular-mean heading smoothing removes per-frame jitter without adding meaningful lag.
- A precise VPS origin makes the 3 m corridor test and 12 m look-ahead physically meaningful.

Known limitations / failure modes:

- **Coverage.** VPS needs Street-View-mapped surroundings and a camera pointed at the world. Indoors, tunnels, dark or featureless scenes degrade or lose the fix.
- **Battery/compute.** Continuous camera + CV + TTS is heavier than a passive GPS read.
- **Accuracy is not consulted.** The code reads `cameraGeospatialPose` but never checks `horizontalAccuracy` / `headingAccuracy`. It trusts any `TRACKING` pose equally.

Concrete hardening opportunities.

1. **Reject low-confidence fixes.** Skip the map/guide update (or soften cues) when `cameraGeospatialPose.horizontalAccuracy` exceeds, say, 2 m, instead of trusting every `TRACKING` sample.
2. **Remove the vestigial scalar `avgHeading accumulator`** in the renderer to avoid confusion; the circular-mean local is the value actually used.
3. **Move the hard-coded API keys** (Directions, Geoapify) out of source into a secrets/config file — they are currently checked into the repository.
4. **Consider gating on `headingAccuracy`** before speaking turn cues, since heading drives the left/right decision.

15 Appendix: end-to-end data-flow trace

One position sample, from photons to speech:

1. Camera frame + IMU delivered to ARCore session (VPS enabled).
2. ARCore localizes visually against Google's 3D world model
-> earth.cameraGeospatialPose { lat, lon, heading, accuracy }.
3. onDrawFrame gate: proceed only if earth.trackingState == TRACKING.
4. Heading pushed into 29-sample deque; circular mean computed
avgHeading = atan2(Sigma sin theta, Sigma cos theta), folded to +/-180.
5. MapView.updateMapPosition: green marker to (lat,lon), rotated to avgHeading; camera bearing locked to heading when navigating.
6. Once per ~second (curFrame == 29) and if a route is set:
ArrowAlignedGuide.update(user=(lat,lon), heading=avgHeading, path):
 - a. advanceSegment -> current route segment index
 - b. lookAheadPoint -> aim 12 m ahead along the polyline
 - c. relativeBearing -> signed angle: +right / -left
 - d. crossTrack (Heron) + isLeftOfSegment -> on-route? which side?
 - e. announceTurnThresholds -> "turn left in 5 meters" (debounced)
7. SpeechGuide.speak(...) -> Android TTS, nav-guidance audio usage, flush for urgent cues, queue for routine cues.
8. UI thread: status text updated; "Arrived" under 1.5 m; "Veering off" snackbar when off-route.

The accuracy the user noticed comes entirely from stage 2 — visual localization replacing GPS. Stages 3–8 are the engineering that turns that precise, stable pose into trustworthy spoken walking guidance.